Buscador de duplicados

Documentación del proceso de creación y sus fases

Cómo pasé de un script de 150 líneas a una aplicación de escritorio para Windows con tres funciones (duplicados exactos, fotos parecidas y reunir imágenes), interfaz en dos niveles y un historial para deshacer cualquier operación.

Aplicación de escritorio para Windows · Python + CustomTkinter
Documento generado el 21/10/2026

Índice

1. Punto de partida . . . 3

1.1. Los principios que me marqué para toda la app . . . 3

2. Stack tecnológico y decisiones . . . 4

2.1. Arquitectura: núcleo separado de la interfaz . . . 4

3. Fases de desarrollo . . . 4

Fase 1 — Núcleo sin interfaz . . . 4

Fase 2 — Ventana principal y escaneo . . . 5

Fase 2b — Filtro por tipo de archivo . . . 5

Fase 3 — Decidir qué conservar . . . 5

Fase 4 — Mover a cuarentena . . . 5

Fase 5 — Empaquetado . . . 5

Ampliación — Rediseño visual . . . 6

Ampliación — Fotos parecidas . . . 6

Ampliación — Previsualización y visor . . . 6

Ampliación — Modo Sencilla / Avanzada . . . 6

Ampliación — Reunir imágenes e historial . . . 6

4. Pruebas . . . 7

5. Problemas que encontré y cómo se resolvieron . . . 8

6. Estado final . . . 8

7. Calidad y publicación . . . 9

8. Qué he aprendido y decisiones mías . . . 9

1. Punto de partida

Siempre he guardado copias de seguridad de mis dispositivos antiguos, como teléfonos u ordenadores, en discos duros externos de varios teras. Con el tiempo, muchos archivos acaban duplicados en subcarpetas, y eso termina comiéndose el espacio del disco. Revisarlo a mano es tedioso, y se vuelve incluso un lío cuando ya no sabes qué carpetas has revisado y cuáles no.

Mi idea de partida era poder lanzar órdenes rápidas y sencillas que compararan imágenes y otros tipos de archivos y me dijeran enseguida si había duplicados o no. Descubrí que la forma más rápida de conseguirlo era usar **Python**.

Empecé con un script sencillo de unas 150 líneas, **buscar_duplicados.py**: le introducía la ruta de una carpeta, la recorría de forma recursiva, calculaba un hash SHA-256 del contenido de cada archivo y me decía si había archivos duplicados y cuántos había analizado, además de generar un CSV con los grupos idénticos. Era seguro (solo leía), pero tras usarlo varias veces me di cuenta de que era poco práctico: tener que escribir la ruta cada vez era lento y no me ayudaba a decidir qué hacer con los duplicados.

Así que decidí crearle una **interfaz** para que fuese más fácil de usar. Al construirla vi que podía añadirle funciones que adaptaran mejor la búsqueda a lo que yo quería, y de ahí pasó de ser un script a un **programa de escritorio completo** para Windows, con varias funciones y opciones de filtrado, empaquetado como un único ejecutable que no exija instalar Python y manteniendo la filosofía de seguridad del script original.

Objetivo del proyecto. Además de resolver esa necesidad, este proyecto es un **ejercicio de aprendizaje**: lo hice para profundizar en Python partiendo de un **nivel básico**, enfrentándome a un problema real de principio a fin (algoritmos, hilos, interfaz gráfica, pruebas, integración continua y empaquetado). Lo hice con Claude como copiloto de programación, y este documento y la guía del código existen para poder aprender de lo construido.

1.1. Los principios que me marqué para toda la app

- **Nunca se borra ni se modifica un archivo original.** La app solo lee y, cuando el usuario lo confirma, *mueve* archivos a una carpeta aparte.
- **Todo es local.** Sin conexión a internet, sin servidor, sin base de datos. Lo que se guarda (configuración e historial) va a la carpeta del usuario en %APPDATA%.
- **Reversible.** Cada operación que mueve o copia archivos deja un registro CSV y una entrada en el historial, y se puede deshacer.

2. Stack tecnológico y decisiones

Elemento	Elección	Motivo
Lenguaje	Python 3.12	Reutiliza la lógica probada del script; multiplataforma; rápido de iterar.
Interfaz	CustomTkinter 5.2	Aspecto moderno sobre Tkinter, 100% local, liviano, fácil de empaquetar.
Hash de contenido	BLAKE2b (librería estándar)	Más rápido que SHA-256 y de sobra fiable para detectar archivos idénticos.
Fotos parecidas	dHash + firma de color (con Pillow)	Sin dependencias pesadas (nada de OpenCV ni redes neuronales).
Empaquetado	PyInstaller 6	Genera un único .exe sin consola que corre sin Python instalado.

2.1. Arquitectura: núcleo separado de la interfaz

Desde el principio decidí separar la **lógica** (paquete `core/`) de la **interfaz** (paquete `ui/`). El núcleo no importa nada de la interfaz y se puede probar entero por consola, sin abrir ninguna ventana. Eso me permitió detectar los errores de lógica pronto, sin mezclarlos con errores de la interfaz.

Módulo de core/	Responsabilidad
hashing.py	hash parcial (primeros 64 KB) y completo; formatear tamaños.
scanner.py	escaneo en 3 pasadas: tamaño → hash parcial → hash completo.
filtros.py	qué archivos incluir/excluir (por tipo, carpeta, nombre, tamaño).
seleccion.py	qué copia se conserva y cuáles se mueven en cada grupo.
quarantine.py	mover los descartados a cuarentena, con CSV de registro.
perceptual.py	detección de fotos parecidas (dHash + color, BK-tree).
mover.py / reunir.py	mover o copiar lotes de archivos; «Reunir imágenes».
historial.py	registro persistente de operaciones + deshacer.
config.py	configuración mínima persistente (recuerda el modo).

3. Fases de desarrollo

Desarrollé el proyecto siguiendo un plan por fases. Cada fase se probaba de forma aislada antes de pasar a la siguiente.

Fase 1 — Núcleo sin interfaz

- Pasé la lógica de escaneo a `core/`, sin interfaz.
- Cambié el hash a **BLAKE2b** con estrategia en dos niveles: primero un hash de los primeros 64 KB (rápido, descarta casi todo), y solo si coincide se calcula el hash del archivo completo.

- Organicé el escaneo en tres pasadas: (1) agrupar por tamaño en bytes — un tamaño único no puede tener duplicados; (2) hash parcial; (3) hash completo. Solo la última pasada lee archivos enteros.

Nota. Para asegurarme de que el núcleo nuevo se comporta igual que el script original, añadí la prueba `comparar_con_original.py`, que construye un árbol de carpetas con trampas (mismo tamaño y distinto contenido, archivos que solo difieren al final, archivos vacíos...) y compara los grupos detectados por ambos. Resultado: idénticos.

Fase 2 — Ventana principal y escaneo

- Añadí el selector de carpeta, una barra de progreso en dos etapas y una tabla de resultados.
- El escaneo corre en un **hilo aparte** para que la ventana no se congele; la comunicación entre hilos va por una cola que la interfaz revisa periódicamente.
- La tabla muestra cada grupo de duplicados como un nodo desplegable con sus archivos dentro.

Fase 2b — Filtro por tipo de archivo

Quería poder descartar los archivos que no me interesan (temporales, extensiones inútiles...), así que añadí al panel «Opciones de escaneo» la posibilidad de **buscar solo ciertos tipos** (casillas de categoría: Imágenes, Audio, Vídeo, Documentos, Comprimidos, que rellenan una lista de extensiones) y además **ignorar** basura (temporales, `node_modules`, `Thumbs.db`, archivos de 0 bytes...). Lo que no pasa el filtro ni se lee ni se hashea, así que también acelera el escaneo.

Fase 3 — Decidir qué conservar

- Cada grupo empieza con todas sus copias en «conservar» (no se movería nada).
- **Regla automática por carpeta preferida**: se indica una carpeta y, en cada grupo, si hay una copia dentro de ella se conserva esa y se marcan las demás para mover.
- **Validación**: el botón de cuarentena permanece bloqueado hasta que cada grupo tiene exactamente una copia marcada como «conservar».

Fase 4 — Mover a cuarentena

- Se elige una carpeta de cuarentena (debe estar fuera de la carpeta escaneada; la app lo comprueba).
- Los archivos marcados se **mueven** preservando su estructura de subcarpetas, para poder localizarlos y devolverlos si hace falta.
- Si en el destino ya hay un archivo con ese nombre, el nuevo se guarda como `nombre (1).ext` — nunca se pisa nada.
- Al terminar se escribe un CSV con cada movimiento.

Fase 5 — Empaquetado

- PyInstaller genera `dist/duplicados.exe`: un único archivo, sin consola, que corre en Windows sin Python.
- El icono de la app es una lupa **dibujada por código** (sin archivos de imagen) y convertida a `icono.ico` multi-tamaño con la librería estándar.

Ampliación — Rediseño visual

Quise que la app resultara más atractiva, así que rehice la interfaz: cabecera con título, secciones numeradas con iconos (glifos Unicode, sin imágenes ni coste de tamaño), tarjetas, un panel de estadísticas con el resultado del escaneo, y un «estado vacío» ilustrado en lugar de una tabla en blanco. Toda la paleta de colores se ajustó para cumplir el contraste mínimo de accesibilidad.

Ampliación — Fotos parecidas

Me preguntaba si se podrían agrupar las fotos «parientes»: la misma foto con menos calidad, con un filtro... Añadí una segunda pestaña que detecta la **misma foto guardada varias veces**: recomprimida, en menor resolución o con un filtro de color. Por cada imagen calcula un **dHash de 64 bits** (mide la estructura, muy estable frente a recompresión y filtros) y una **firma de color**. Un BK-tree busca vecinos por distancia de Hamming sin comparar todas contra todas, y un Union-Find une las cadenas de parecidos. Elegí que **no** detecte recortes ni rotaciones, porque haría falta un motor de visión mucho más pesado.

Ampliación — Previsualización y visor

Para decidir con seguridad quise ver lo que iba a mover:

- Miniatura en cada fila de la tabla (imagen real o un recuadro con la extensión, coloreado por categoría).
- Panel de vista grande a la derecha, con nombre, ruta, tamaño, fecha y resolución.
- Al pulsar la previsualización se abre un **visor a pantalla casi completa** con zoom (rueda del ratón) y desplazamiento arrastrando.

Ampliación — Modo Sencilla / Avanzada

Como ya había muchas opciones, decidí que la app arrancara en una versión simplificada, con un interruptor en la cabecera. Arranca en **Sencilla** (solo elegir carpeta, escanear y mover; la app preselecciona sola la copia principal de cada grupo) y recuerda la elección. En **Avanzada** aparece todo: opciones de escaneo, carpeta preferida, exportar CSV, deslizador de sensibilidad, historial...

Ampliación — Reunir imágenes e historial

- Añadí una tercera pestaña que junta en una carpeta común las fotos repartidas por una carpeta. Estrategias «por archivos» (preservando subcarpetas) o «carpeta entera»; mover o copiar.
- Diálogo para decidir qué hacer con los archivos que no son imágenes: dejarlos, moverlos, o agruparlos por tipo en una carpeta con el nombre que elija el usuario.
- **Historial** (botón en la cabecera, modo Avanzada): lista todas las operaciones de mover/copiar y permite **deshacer** cada una — al mover, devuelve los archivos; al copiar, borra las copias con confirmación.

4. Pruebas

Pruebo el proyecto con `pytest`: 128 pruebas (15 suites y 113 unitarias). No hay ningún test que «finja» el sistema de archivos: cada uno construye archivos de verdad en una carpeta temporal, ejecuta la lógica real y comprueba el resultado en disco.

Suite	Qué comprueba
comparar_con_original	El núcleo detecta los mismos duplicados que el script original.
probar_filtros	Lista blanca por categoría + lista negra + desglose por extensión.
probar_seleccion	Regla de carpeta, ajuste manual, preselección de la copia principal.
probar_cuarentena	Mover de verdad, log CSV, colisiones, casos inválidos.
probar_cuarentena_carpetas	Se mueven los ARCHIVOS, nunca las carpetas.
probar_perceptual	Agrupación de fotos parecidas; corrupta no rompe nada.
probar_reunir	Estrategias por archivos / carpeta entera; reglas por tipo.
probar_historial	Registrar, listar y deshacer (mover y copiar).
humo_ventana / humo_parecidas / humo_sencilla	Montaje de la ventana y flujos de punta a punta.
humo_cuarentena_ui / humo_reunir_ui	Pulsando los botones reales, con diálogos simulados.
humo_registro / humo_dialogos	Errores inesperados en el registro; visor, historial, diálogos, CSV y miniaturas.
test_contraste (71)	Contraste WCAG AA de cada pareja de texto y fondo, en los dos temas.
test_scanner_pasadas / test_registro / test_unidades	Cada pasada del escáner, el registro de errores, la configuración, el CSV y la versión.

5. Problemas que encontré y cómo se resolvieron

Problema	Solución
CustomTkinter reaplicaba su propio icono 200 ms después de abrir la ventana.	Fijar el icono con iconbitmap(.ico) en vez de iconphoto, para que la comprobación interna de la librería lo detecte y no lo pise.
Pasar varias imágenes a la vez a iconphoto() desdibujaba el icono.	Usar una sola imagen bien elegida, o el .ico multi-tamaño en el .exe.
La resolución de las fotos salía mal en el visor y la lista.	Leer im.size ANTES de im.draft(): draft reduce la imagen y cambia el tamaño reportado.
El «estado vacío» no tapaba la galería (CTkScrollableFrame).	Alternar con grid()/grid_remove() en vez de tkraise().
Pillow inflaba el .exe de 13 a 20 MB.	Asumí el coste al añadir fotos parecidas y miniaturas; Pillow es necesario y va en requirements.txt (no así reportlab, que solo genera esta documentación).
Una duda mía al probarlo: «¿mueve las carpetas en vez de las fotos?».	Dos tests nuevos confirmaron que siempre se mueven archivos individuales; lo que se ve es la estructura recreada en el destino y las subcarpetas de origen que quedan vacías.
Tras mover a cuarentena, la vista previa conservaba la imagen anterior bajo el texto «selecciona un archivo».	CTkLabel.configure(image=None) no la quita: hay que limpiar el Label interno de tkinter.
Con muchas fotos parecidas, la galería se cortaba y la barra de desplazamiento desaparecía.	Windows no dibuja ventanas de más de 32.767 px de alto. Ahora los grupos se reparten en páginas (Anterior / Siguiente) y una prueba comprueba que ninguna se acerca a ese límite.
Mis pruebas de ventana escribían en el historial real de la app.	Ahora usan un APPDATA temporal, y una comprobación común avisa si alguna vuelve a tocar los datos reales.
Dos pollers leyendo la misma cola se robaban mensajes.	Colas separadas para las miniaturas y para la operación en curso.

6. Estado final

Métrica	Valor
Código Python (sin build ni dist)	~8.800 líneas
· núcleo (core/)	~2.100 líneas, 13 módulos
· interfaz (ui/)	~3.500 líneas, 13 módulos
· pruebas	~3.000 líneas: 15 suites + 113 unitarias
Dependencias de la app	customtkinter, pillow
Ejecutable final	duplicados.exe (~20 MB, un solo archivo) o ZIP con carpeta (~20 MB), sin consola
Estado de las pruebas	128/128 en verde; cobertura: núcleo 93 %, interfaz 79 %

La aplicación cubre tres necesidades relacionadas — encontrar duplicados exactos, encontrar fotos parecidas y reunir imágenes dispersas — y en las tres mantuve la misma regla: nunca borra tus originales y todo se puede deshacer.

7. Calidad y publicación

Antes de publicar el proyecto en GitHub dediqué una tanda de trabajo a la calidad, siempre con las pruebas como red de seguridad:

- **Estructura.** La ventana principal tenía más de 2.100 líneas en una sola clase. Ahora es una carcasa de ~200 líneas y cada pestaña es un componente con su propio estado, en su módulo (`pestana_*.py`). Lo hice pestaña a pestaña, comprobando cada vez que las 16 capturas de la interfaz (tema claro y oscuro) seguían siendo idénticas píxel a píxel. También repartí `escanear()` en una función por pasada.
- **Herramientas.** pytest (con cobertura del núcleo y de la interfaz), ruff para el estilo y mypy para los tipos del núcleo, que está completamente anotado. Todo se ejecuta en cada *push* con GitHub Actions, y Dependabot avisa de actualizaciones.
- **Contraste.** Las pruebas miden con la fórmula de WCAG cada pareja de texto y fondo en el tema claro y en el oscuro (nivel AA, 4,5:1). Con ellas sustituí los azules y grises por defecto de CustomTkinter, que con texto blanco no llegaban, por colores propios (5,7:1).
- **Registro de errores.** Sin consola, los errores del .exe desaparecían. Ahora se anotan en un archivo y la app avisa de dónde está.
- **Rendimiento.** `pruebas/benchmark.py` compara el escaneo en 3 pasadas con hashear todo: lee un 66 % menos de bytes y encuentra exactamente los mismos grupos.
- **Dos formatos de descarga.** Con Avast instalado, el .exe de un solo archivo se me quedó congelado en el primer arranque (se descomprime en una carpeta temporal cada vez). Por eso cada versión incluye también un ZIP con carpeta, que no extrae nada al arrancar. Al probarlo vi que, sin consola, una simple advertencia de CustomTkinter (ruta de más de 260 caracteres) hacía caer la app; ahora se descartan las salidas que no existen.
- **Privacidad en capturas.** Las capturas y el GIF se hacen sobre una carpeta de ejemplo con ruta neutra, para que no aparezca el nombre de usuario de Windows.

Cómo lo hice. Lo desarrollé con Claude (Anthropic) como copiloto de programación: yo decidía qué construir y revisaba el resultado, y Claude escribía y probaba el código en sesiones de trabajo conjunto. Los commits llevan la línea «Co-Authored-By» para dejarlo claro.

8. Qué he aprendido y decisiones mías

- **Lo que más me costó:** cómo dividir el código para que no quedaran archivos demasiado largos. Se nota en la estructura actual: la lógica (`core/`) separada de la interfaz (`ui/`) y un componente por pestaña.
- **Lo que más he aprendido:** la versatilidad de Python para ejecutar scripts directamente sobre los archivos: con muy pocas líneas se puede recorrer un disco, comparar contenidos y mover archivos.

- **Decisiones mías:** que tuviera tema **claro y oscuro**, la propia **interfaz** (el script original era de consola) y el **modo Sencilla y Avanzada**.

No hay una hoja de ruta cerrada: iré mejorando la aplicación sobre la marcha, según vea con la práctica qué funciones le faltan. Si alguien tiene ideas para funcionalidades futuras, me encantará conocerlas.